Volatility Forecasting Engine — Technical Reference (Days 8–14)

From the target and features through the trained challenger model.

This document explains everything built in Days 8–14 of the project so that, after reading it alongside the code, you understand every moving part — including the finance, which is written assuming **no prior finance background**.

- Days 8–9: the label (target) and the features.
- Day 10: how we validate without cheating (walk-forward + purge/embargo).
- Day 11: how we score forecasts (metrics).
- Days 12–14: the models — simple baselines, GARCH, and the gradient-boosting challenger.
- Day 15 (registry) is **not yet built**; there is a short "what's next" section.

0. The big picture

Everything is one pipeline that turns raw prices into a volatility forecast:

```
raw prices + macro  ->  FEATURES (what we know at day t, looking back)
                    ->  TARGET   (what we want to predict: vol of the next 21 days)
                    ->  VALIDATE (train on the past, test on the future, no leakage)
                    ->  SCORE    (MAE, RMSE, QLIKE, calibration)
                    ->  MODELS   (baselines, GARCH, gradient boosting) -> MLflow
```

The single most important idea: on each day *t* we split the world in two. **Features** use only information available **up to and including day t** (the past). The **target** uses only information **after day t** (the future). The model learns the map from the past-description to the future-answer, and we are obsessive about never letting the future leak into the past.

1. Finance primer (plain language)

Every term below is a statistics idea with a finance name.

Price vs. adjusted close. A stock's raw closing price jumps artificially on stock splits and dividend payments. The **adjusted close** corrects for those so the series is continuous. We use adjusted close for *all* return math; using raw close would create fake overnight crashes.

Return. How much the price changed. A **log return** is `ln(price_today / price_yesterday)`. We use logs because they add up cleanly across time and behave better statistically. A day's log return of `0.01` ~ a 1% move.

Volatility. How much a price bounces around — the *size* of moves, regardless of direction. Formally the standard deviation of returns. High volatility = wild swings; low = calm.

Realized volatility. Volatility *measured* from what actually happened over a window (e.g. the last 21 days). It is a measurement, not a forecast.

Annualization. Daily volatility is scaled to a yearly number by multiplying by `sqrt(252)` (there are ~252 trading days per year). So a daily vol of 1.3% becomes `0.013 * sqrt(252) ~ 21%` "annualized." Everyone quotes vol annualized.

Why forecast volatility and not price? The *direction* of tomorrow's price is almost random (unpredictable). But the *size* of moves is predictable, because volatility **clusters** (turbulent days follow turbulent days) and **mean-reverts** (spikes decay back toward normal). That is the entire reason this project exists, and we proved it visually on Day 7.

The VIX. A single number published daily that is the market's *expectation* of S&P 500 volatility over the next 30 days — "the fear index." When investors panic it spikes (it hit ~80 in March 2020). Because it is a forward-looking estimate, it is one of our strongest input features.

Treasury yields and the term spread. A Treasury is a loan to the US government; the **yield** is the interest rate it pays. We use the 10-year yield (`DGS10`) and the 3-month yield (`DGS3MO`). Their difference, the **term spread** (10y - 3m), is a slow-moving signal of the economic climate.

Variance risk premium (VRP). The gap between implied vol (VIX) and realized vol. Usually positive (implied exceeds realized) and historically informative.

Leverage effect. Volatility tends to rise *more* after negative returns than after positive ones. We encode this with return-based features.

2. Day 8 — The target and the feature contract

2.1 What the target is

For each stock on each day *t*, the **label** the model predicts is the **annualized realized volatility of the next 21 trading days** (a trading month). In words: "standing on day *t*, how wild will this stock be over the coming month?"

The exact formula we locked in:

```
target(t) = sqrt( sum of squared daily log returns over days t+1..t+21  x  252/21 )
```

- We sum the *squared* returns (variance is about squared moves), scale by `252/21` to annualize, and take the square root to turn variance into volatility.
- We **store the level** (e.g. 0.21 = 21%) in the database, but the model is trained on its **logarithm** (volatility is positive and right-skewed; its log is closer to a bell curve, which helps the model), and we exponentiate back at the end.

Two consequences that matter: - The label for day *t* cannot be known until the close of day *t+21*. We store that date as **`target_available_on`** — it is the key the monitoring layer will later use to match a forecast to its eventual outcome. - The **most recent 21 rows** of every stock have **no label** (the future hasn't happened). They are excluded from training but still receive predictions.

2.2 Why the sum-of-squares formula (decision)

There were two valid formulas. We chose sum-of-squared-returns (rather than the sample standard deviation) because it is the standard "realized variance" convention *and* it matches how our HAR-RV baseline defines the same quantity — so the challenger and the baseline are measuring the exact same thing. Fair fight.

2.3 `target.py` — important functions

- **`forward_realized_vol(adj_close, horizon=21, trading_days=252)`** — the core. For one stock's price series it: computes log returns, squares them, uses a rolling 21-day sum **shifted backward by 21** so the window covers the *future* (this shift is the whole trick that makes the label forward-looking), annualizes, square-roots, and also returns `target_available_on`. The last 21 rows automatically come out null.
- **`build_targets(prices, ...)`** — runs the above **per stock** and stacks the results, so no label ever bleeds across stocks.

2.4 The feature contract

`config/feature_contract.yml` is a version-controlled **declaration** of every feature (name, group, definition, window) with a `feature_version: v1`. It is the single source of truth used by the feature build (Day 9) and later by the promotion gate. `contract.py` loads it into typed objects:

- **`FeatureSpec`** — one feature's metadata.
- **`FeatureContract.feature_names()`** — the feature names in order (with an option to exclude "optional" features like the calendar ones).
- **`load_feature_contract()`** — parses the YAML.

2.5 The proof (test)

The done-when was a **synthetic closed-form test**: build a fake price series whose daily return is a constant `c`, so its annualized vol is provably `|c|*sqrt(252)`. We engineered `c` so the answer is exactly 0.20 and confirmed the code returns 0.20, that the last 21 rows are null, and that `target_available_on` is the 21st future date.

3. Day 9 — Building the features

The features describe the *state* of each stock on day *t*, looking only backward. We compute ~24 of them. Here is what each group means in plain language.

Feature(s)	What it captures
`log_rv_1, _5, _21, _63, _126`	Realized volatility over the last 1/5/21/63/126 days (the "HAR ladder"): recent vs. longer-term vol. The core signal.
`log_ewma_vol`	Exponentially-weighted vol — a smoother, faster-adapting version that weights recent days more.
`rv_ratio_5_21, rv_ratio_21_126`	Short-window ÷ long-window vol. Tells the model whether the stock is currently *above or below* its own normal level (mean-reversion signal).
`log_parkinson_21`	A vol estimate that uses the day's high–low **range**, which is more information-efficient than close-to-close.
`ret_1, ret_5, ret_21`	Cumulative returns over 1/5/21 days.
`abs_ret_1`	Size of the most recent move (encodes the leverage effect).
`log_downside_vol_21`	Volatility computed from **negative** days only (downside risk).
`volume_z_21`	How unusual today's trading volume is vs. its recent norm (volume spikes often precede vol).
`log_vix`	The VIX (market's implied vol), in logs.
`vix_chg_5`	5-day change in the VIX.
`vrp_21`	Variance risk premium proxy: VIX minus the stock's own realized vol.
`log_index_rv_21`	Realized vol of the market (SPY).
`rv_ratio_to_index_21`	Stock vol ÷ market vol (is this stock calmer or wilder than the market?).
`term_spread`	10-year minus 3-month Treasury yield (macro regime).
`entity_mean_log_rv`	The stock's own average vol so far (an **expanding** average — never uses the future). Lets one shared model learn that some stocks are structurally more volatile.
`dow, month`	Day-of-week and month (weak calendar effects; optional).

3.1 Leakage rules baked in

- Every feature uses only data up to and including day *t*.

- `entity_mean_log_rv` uses an **expanding window** (all past days, growing over time) rather than the full-sample average — otherwise it would secretly encode the future.

3.2 `compute.py` — important functions

- **`_log_returns`, `_safe_log`, `_realized_vol`** — small helpers. `_safe_log` maps non-positive inputs to "missing" instead of `-infinity`; `_realized_vol` computes annualized vol over a trailing window.
- **`compute_ticker_features(prices)`** — all the per-stock features above from one stock's OHLCV.
- **`compute_market_features(vix, dgs10, dgs3mo, spy_adj_close)`** — the market-wide features (VIX, term spread, index vol), aligned to trading days and forward-filled across small holiday gaps.

3.3 `build.py` — the orchestration

- **`build_feature_frame(ohlcv, macro)`** — joins per-stock features with the market features, computes `vrp_21` and `rv_ratio_to_index_21`, attaches the Day-8 target, then **drops early rows** that lack enough history (up to the 126-day window) rather than filling them in, and **keeps** the recent unlabeled rows.
- **`_upsert_features(...)`** — writes rows to the `features` table with an idempotent "insert or update," so re-running never duplicates.
- **`build_features(engine)`** — the full load -> compute -> write path.

`dictionary.py` auto-generates `assets/feature_dictionary.md` from the contract, and migration `0002_add_feature_columns.sql` added the ~24 columns (Day 2 had deliberately left them out until the contract existed).

3.4 A real bug we caught

Pandas quietly turns `None` back into `NaN` inside a float column, and Postgres `numeric` stores `NaN` as a special value — so the recent rows were saved as `NaN` instead of true `NULL`. The integration test caught it; we fixed it by converting to object dtype before writing. Result: **117,364 feature rows**, 29 stocks, with 607 correctly-null recent labels and zero `NaN`.

4. Day 10 — Validation without cheating

4.1 Why not a random train/test split

In normal machine learning you shuffle rows into train and test. **Here that would cheat.** Our label for day *t* covers days *t+1..t+21*, so it overlaps neighboring rows. A random split would put day *t* in training and day *t+3* in testing, and they share almost the same future — the model would "see the answer." So we split along **time**.

4.2 Walk-forward with purge and embargo

- **Walk-forward (rolling origin):** always train on the past, test on a later window, step forward, repeat. The training window expands with each fold.
- **Purge (21 days):** drop the training days whose forward label would reach into the test window.
- **Embargo (5 days):** an extra safety gap after the purge.

So there is always a gap of `purge + embargo = 26` trading days between the last training day and the first test day — larger than the 21-day label horizon, which guarantees no training label overlaps the test period.

4.3 `splitter.py` — important pieces

- **`WalkForwardConfig`** — the parameters (`n_splits`, `test_size`, `purge`, `embargo`, `min_train_size`), loaded from `config/validation.yml`.
- **`Fold`** — one fold: the training row indices, the test row indices, and the date boundaries.
- **`walk_forward_folds(trade_dates, config)`** — builds the folds: it lays `test_size`-day test windows end-to-end at the recent part of the sample and trains on everything up to `purge+embargo` days before each.

4.4 The non-negotiable leakage test

For every fold we assert **max(train index) < min(test index)** (with the panel sorted by date) and that the gap exceeds the 21-day horizon. This test must never be skipped or weakened — it is what certifies the whole project isn't secretly cheating.

5. Day 11 — Scoring forecasts (metrics)

All metrics take predicted and actual **annualized vol levels**.

- **MAE (mean absolute error):** average size of the miss, in vol points. Simple and interpretable. Lower is better.
- **RMSE (root mean squared error):** like MAE but punishes big misses more.
- **QLIKE (quasi-likelihood):** the **primary** metric for volatility. It compares *variances* and is robust to the fact that "true" volatility is itself a noisy measurement. Formula per point: `v_true/v_pred - ln(v_true/v_pred) - 1`, averaged. A perfect forecast scores 0; lower is better. QLIKE punishes **under-forecasting** volatility more than over-forecasting — appropriate for risk work.
- **Mincer–Zarnowitz (MZ):** a calibration check. We regress the actual vol on the predicted vol; a well-calibrated model has **slope ~ 1** (on average it's neither systematically too high nor too low) and a higher **R^2**.

5.1 `metrics.py` — important functions

- **`mae`**, **`rmse`**, **`qlike`** — the three error metrics.
- **`mincer_zarnowitz`** — returns the calibration slope and R^2.
- **`compute_metrics`** — computes all of them at once into a `Metrics` bundle.
- **`_clean_pair`** — drops non-finite or non-positive pairs before scoring.

We verified each metric against a hand-computed value (e.g. QLIKE of a perfect forecast is exactly 0; a known line gives MZ slope exactly 2).

6. Days 12 — Baseline models and experiment tracking

Baselines are simple models the fancy model must beat. If it can't beat these, it isn't worth the complexity. Each model is just a function `(train, test) -> predicted vols`.

6.1 The baselines (finance meaning)

- **Persistence:** "tomorrow looks like today" — predict next month's vol as the current trailing 21-day realized vol. A naive but surprisingly decent baseline because vol is persistent.
- **EWMA (exponentially weighted moving average):** like persistence but with a smoothly-decaying weight on older days (the RiskMetrics industry standard).
- **HAR-RV (Heterogeneous AutoRegressive model of Realized Volatility):** a simple linear regression that predicts future vol from the stock's vol over the last **day, week (5d), and month (21d)**. It captures that traders operate on different horizons. Famously hard to beat — it is *the* benchmark in the volatility literature. We fit it per fold on training data only.

`baselines.py` implements `persistence`, `ewma`, `har_rv`, and bundles them in `BASELINE_MODELS`.

6.2 MLflow — the experiment tracker

MLflow is a lab notebook + filing cabinet for model runs. Every evaluation logs its parameters, metrics, and git commit to a server (running in Docker at `http://localhost:5001`) so results are reproducible and comparable.

`tracking.py` — important functions: - **`configure_mlflow()`** — points MLflow at the server (URL from `.env`). - **`git_sha()`** — the code version, tagged on every run (ties results to code). - **`log_walk_forward(...)`** — logs one model as a **parent run** with aggregated mean metrics and a **nested child run per fold**. - **`mean_metrics(...)`** — averages each metric across folds.

6.3 `evaluate.py` — the shared evaluation loop

- **`load_labeled_panel(engine, feature_cols)`** — loads the labeled feature rows (target not null), sorted by date.
- **`evaluate_model(panel, folds, model_fn)`** — runs one model across all folds and returns per-fold metrics.
- **`run_evaluation(engine, models, experiment)`** — evaluates several models, logs them to MLflow, and returns a comparison table sorted by QLIKE.

7. Day 13 — GARCH

7.1 What GARCH is (plain language)

GARCH = *Generalized AutoRegressive Conditional Heteroskedasticity*. Long name, simple idea: it is a classic econometrics model that says **today's variance depends on yesterday's variance and yesterday's surprise (squared return)**. It formalizes volatility clustering directly. Think of it as "ARIMA for variance." It is the standard econometric tool, so including it signals domain competence — and it turned out to be a very strong baseline here.

A GARCH(1,1) has three parameters: a baseline variance, how much weight to put on the last squared return (the "shock"), and how much on the last variance (the "memory"). We fit it **per stock** on the training returns, then roll it forward to forecast the next 21 days' variance and annualize it.

7.2 `garch.py` — important function

- **`garch(train, test)`** — for each stock: fit GARCH(1,1) on the training returns only (so the test period never influences the parameters — no leakage), fix those parameters, run the variance recursion over the test returns (which are known at prediction time), and produce a 21-day-ahead annualized vol forecast at each test date. If a fit fails to converge, it falls back to persistence.

Returns are scaled to percent for the optimizer's numerical stability, then un-scaled. The same `(train, test) -> preds` signature means GARCH plugs into the exact same evaluation loop as everything else.

8. Day 14 — The gradient-boosting challenger

8.1 What it is (plain language)

Gradient boosting builds many small decision trees in sequence, each one correcting the previous ones' mistakes. It is the workhorse for tabular data: strong, fast, no GPU, and it handles messy features and interactions automatically. We use scikit-learn's `HistGradientBoostingRegressor`. Deep learning would be overkill (and forbidden by the project rules) on a few thousand rows per stock.

Two design choices: - **One pooled model across all stocks** (not 30 separate models), with the `entity_mean_log_rv` feature so it can still learn that some stocks are structurally more volatile. This gives ~30x more training data, one artifact to deploy, and instant generalization to a new stock. - It is trained on the **log** target and exponentiated back to a level.

8.2 `pipeline.py` — important functions

- **`build_pipeline(params)`** — a scikit-learn Pipeline: median imputation -> gradient boosting. (Trees don't need scaling or outlier-clipping, so we skip those; they matter only for the linear baselines.) The pipeline is fit **per fold on training data only** — this is how the "no preprocessing on validation data" leakage rule is enforced in code.
- **`make_challenger(feature_cols, params)`** — returns a `(train, test) -> preds` function for given hyperparameters, so the challenger evaluates through the same loop as the baselines.

8.3 `challenger.py` — important functions

- **`PARAM_GRID`** — a small, deliberately coarse set of hyperparameter combinations (learning rate, tree size, regularization).
- **`search_challenger()`** — a **time-aware** search: it evaluates each grid config across the walk-forward folds and picks the one with the best mean QLIKE. (We never use random cross-validation — that would leak time.)
- **`_write_comparison(table, best_params)`** — writes the honest write-up to `assets/model_comparison.md`.
- **`_log_prediction_scatter(best_params)`** — logs a predicted-vs-actual scatter plot to MLflow as an artifact.
- **`main()`** — runs the search, evaluates the best challenger against every baseline + GARCH, writes the comparison, and logs diagnostics.

9. Results and honest interpretation

Walk-forward mean metrics (lower QLIKE/MAE/RMSE is better; MZ slope near 1 is better):

model	MAE	RMSE	QLIKE	MZ slope	MZ R^2
challenger (gradient boosting)	**0.0664**	**0.0977**	**0.2406**	1.076	**0.423**
garch	0.0675	0.1016	0.2428	0.759	0.320
ewma	0.0759	0.1135	0.2751	0.578	0.323
har_rv	0.0695	0.1062	0.3234	1.008	0.280
persistence	0.0794	0.1207	0.3387	0.493	0.267

Reading it honestly: - The challenger is **best on every error metric**, best-calibrated among the strong models (slope ~ 1.08), and has the highest R^2 (0.42). - It beats **HAR-RV by ~25.6%** on QLIKE — but only **~0.9%** over **GARCH**, which is a genuinely strong baseline here. - The small edge over the *strongest* baseline is the credible story. It is **not** a leakage red flag: the purge/embargo gap and the leakage test guarantee no future information leaked, so the model simply found a modest real edge. - The honest interview framing: "the challenger edges a strong GARCH baseline and clearly beats HAR-RV, and the real contribution is the operational loop around the model (weekly retrain, promotion gate, monitoring)."

10. Decisions log (what we chose and why)

- **Target = sum-of-squared-returns** (not sample std): the realized-variance convention, and it matches HAR-RV's definition so the comparison is fair.
- **Store the vol level, model the log:** volatility is positive and right-skewed; its log is closer to normal, which helps a squared-error learner.
- **Full feature contract up front (`v1`):** one source of truth for the build and the future promotion gate; feature columns added via a forward-only migration.
- **Expanding-window entity encoding:** a full-sample average would leak the future; expanding uses only the past.
- **Drop insufficient-history rows, keep recent unlabeled rows:** don't fabricate features; still allow predictions for the most recent days.
- **Walk-forward + 21-day purge + 5-day embargo:** the only valid way to validate an overlapping-label time series; random splits would leak.
- **QLIKE as the primary metric:** robust to the noisy volatility proxy and appropriate for risk (penalizes under-forecasting).
- **Baselines before the fancy model:** persistence/EWMA/HAR-RV/GARCH set the bar; a model that can't beat them isn't worth its complexity.
- **One pooled gradient-boosting model + entity feature** (not 30 models): more data, one artifact, generalizes to new names.
- **No new abstract classes unless justified:** models are plain functions sharing one signature; only genuinely swappable components (data providers) use an interface.
- **matplotlib added to core dependencies:** so training can log plot artifacts to MLflow.

11. Day 15 — what's next (not yet built)

The model is trained and compared; the *operational* half is next: - **Model registry (Day 15):** register the winning model to a "Production" stage in MLflow so the serving layer loads it by stage, never from a file path. - **Promotion gate (Day 16):** an automated, logged rule that only promotes a new model if it clears QLIKE/calibration/stability/schema checks — the single most "senior" part of the project. - **Serving API, monitoring, dashboard (Days 17+).**

12. Glossary

- **Adjusted close** — price corrected for splits/dividends; used for all returns.
- **Log return** — `ln(today/yesterday)`; additive across time.
- **Volatility** — size of price moves (std of returns); direction-agnostic.
- **Realized volatility** — vol measured over a past window.
- **Annualization** — scaling daily vol by `sqrt(252)`.
- **Target / label** — annualized realized vol of the next 21 trading days.
- **Feature** — a backward-looking descriptor of the stock on day *t*.
- **VIX** — market's implied 30-day volatility ("fear index").
- **Term spread** — 10-year minus 3-month Treasury yield.
- **Variance risk premium** — VIX minus realized vol.
- **Walk-forward** — train on past, test on future, step forward.
- **Purge / embargo** — gaps that stop the label horizon from leaking into the test.
- **Leakage** — future information contaminating the features/training.
- **MAE / RMSE** — average / squared-penalized error of the forecast.
- **QLIKE** — variance-based volatility loss; primary metric; 0 is perfect.
- **Mincer–Zarnowitz** — calibration check (slope ~ 1 is good).
- **Persistence / EWMA / HAR-RV** — simple vol baselines.
- **GARCH** — econometric model where today's variance depends on yesterday's variance and shock.
- **Gradient boosting** — sequential decision-tree model; the challenger.
- **MLflow** — experiment tracking + model registry.

13. How to run it

```
make up          # start Postgres + MLflow (Docker)
make migrate     # apply DB migrations (incl. feature columns)
make backfill    # load ~16 years of prices + macro
make quality     # data quality gate
make features    # build the features table
make baselines   # evaluate persistence/EWMA/HAR-RV -> MLflow
make challenger  # search + compare gradient boosting vs all baselines
```

MLflow UI: `http://localhost:5001`. Comparison write-up: `assets/model_comparison.md`. Feature dictionary: `assets/feature_dictionary.md`.